# Руководство по решению лабораторных работ 1–5 по OpenMP

C++17  директивы  библиотеки  построчный разбор  проверка результатов

Документ содержит готовые решения пяти заданий, объяснение подключаемых библиотек, алгоритмы, полный код с номерами строк, построчный разбор, команды компиляции, ожидаемые результаты и вопросы для защиты. Все программы используют один и тот же принцип: сначала проверяется корректность, затем измеряется параллельное выполнение.

**Главный практический вывод.** OpenMP не делает любой код автоматически быстрым: программист обязан явно распределить работу и устранить гонки данных. Для суммирования и других ассоциативных операций предпочтителен reduction; atomic и critical нужны для более точечной синхронизации.

## Содержание


- 1. Подготовка среды и базовые понятия
- 2. Лабораторная 1  Введение в OpenMP
- 3. Лабораторная 2  Суммирование элементов массива
- 4. Лабораторная 3  Поиск минимума и максимума
- 5. Лабораторная 4  Скалярное произведение
- 6. Лабораторная 5  Вычисление числа π
- 7. Методика измерений и ответы для защиты

# Подготовка среды и базовые понятия

## Что такое OpenMP

OpenMP — набор директив компилятора, функций среды выполнения и переменных окружения для параллельного программирования на системах с общей памятью. Программа начинается одним главным потоком. При входе в parallel-область создается команда потоков, а после выхода выполнение снова продолжается одним потоком.

Директива имеет форму #pragma omp ... . Без ключа поддержки OpenMP компилятор обычно игнорирует pragma, поэтому программа может собраться, но останется последовательной. Ключ -fopenmp одновременно включает обработку директив и подключает библиотеку времени выполнения.

## Библиотеки и заголовочные файлы

| Файл | Зачем подключается |
|---|---|
| <omp.h> | Объявляет omp_get_wtime, omp_get_thread_num, omp_get_num_threads, omp_set_num_threads, omp_get_max_threads и другие функции OpenMP. |
| <iostream> | Ввод и вывод через std::cin, std::cout и std::cerr. |
| <cstdio> | Форматированный вывод printf. В лабораторной 1 один вызов printf уменьшает вероятность перемешивания частей одной строки. |
| <vector> | Динамические одномерные массивы std::vector с автоматическим управлением памятью. |
| <cstddef> | Тип std::size_t для размеров и индексов контейнеров. |
| <algorithm> | std::min, std::max и std::minmax_element для эталонной последовательной проверки. |
| <limits> | Безопасные начальные значения: самый большой int для минимума и самый маленький int для максимума. |
| <iomanip> | Управление количеством знаков при печати времени, ускорения и числа π. |
| <cmath> | std::abs и константа M_PI, если она предоставляется реализацией. |
| <utility> | Поддержка пар и структурированного присваивания результата std::minmax_element. |

## Ключевые директивы и функции

| Конструкция | Назначение |
|---|---|
| parallel | Создает команду потоков. Код внутри выполняет каждый поток. |
| for | Распределяет итерации следующего цикла между уже созданными потоками. |
| parallel for | Сокращенная запись: создать потоки и сразу распределить цикл. |
| num_threads(n) | Просит использовать n потоков только для данной parallel-области. |

| Конструкция | Назначение |
|---|---|
| master | Выполняет блок только поток 0. В конце нет неявного барьера. |
| reduction | Создает локальную копию переменной для каждого потока и безопасно объединяет копии в конце. |
| atomic | Делает атомарной одну следующую операцию обновления общей переменной. |
| critical | Разрешает только одному потоку выполнять именованный блок в конкретный момент. |
| schedule(static) | Делит итерации заранее на приблизительно равные непрерывные части; хорошо подходит для одинаково тяжелых итераций. |
| omp_get_wtime() | Возвращает текущее время в секундах; разность двух вызовов дает длительность участка. |

## Как проверять программы

1. Собрать с оптимизацией -O2 и ключом -fopenmp.
2. Сначала проверить ответ на 10 элементах или на заранее известной сумме.
3. Только после проверки запускать массивы от 1 000 000 элементов.
4. Для времени выполнить несколько повторов; в решениях берется лучший из трех результатов.
5. Не ожидать линейного ускорения: мешают создание потоков, синхронизация, пропускная способность памяти и другие процессы системы.

$$S_p = T_1 / T_p \qquad E_p = S_p / p$$

Здесь $T_1$ — время одного потока, $T_p$ — время p потоков, $S_p$ — ускорение, $E_p$ — эффективность. В коде лабораторных 3 и 4 печатается $S_p$.

# Лабораторная работа 1 Введение в OpenMP

**Цель.** Проверить поддержку OpenMP, увидеть выполнение parallel-области каждым потоком, вывести идентификаторы и сравнить omp_set_num_threads с num_threads.

**Результат.** Программа содержит проверку макроса _OPENMP, одну область с числом потоков по умолчанию и две области с явно заданными размерами команды.

Файл решения: lab1_openmp_intro.cpp

## Алгоритм решения

1. Проверить, определен ли макрос _OPENMP. Его значение имеет формат yyyymm и сообщает дату поддерживаемой спецификации.

2. Открыть parallel-область. Каждый поток получает свой номер через omp_get_thread_num и печатает его.

3. Внутри master вывести omp_get_num_threads, поэтому общее число печатает только поток 0.

4. Установить значение по умолчанию omp_set_num_threads(2).

5. Для первой следующей области указать num_threads(3): локальная опция имеет приоритет.

6. Для второй области не указывать опцию: действует значение 2 из omp_set_num_threads.

**Ответ на вопрос задания.** Программа может содержать произвольное количество последовательных parallel-областей. Практический предел определяется размером программы и ресурсами системы. Вложенные parallel-области также возможны, но их фактическое распараллеливание зависит от настроек вложенного параллелизма.

## Полный код с номерами строк

```
 1 | #include <cstdio>
 2 | #include <omp.h>
 3 |
 4 | int main() {
 5 | #ifdef _OPENMP
 6 |     std::printf("OpenMP is supported. _OPENMP = %d\n", _OPENMP);
 7 | #else
 8 |     std::printf("OpenMP is not enabled during compilation.\n");
 9 |     return 1;
10 | #endif
11 |
12 |     // Запрещаем среде выполнения самовольно уменьшать число потоков.
13 |     omp_set_dynamic(0);
14 |
15 |     std::printf("\n1) Parallel region with the default thread count\n");
16 | #pragma omp parallel
17 |     {
18 |         const int thread_id = omp_get_thread_num();
19 |         std::printf("Thread id: %d\n", thread_id);
20 |
21 | #pragma omp master
22 |         {
23 |             std::printf("The master thread reports %d threads in total.\n",
24 |                         omp_get_num_threads());
25 |         }
26 |     }
27 |
28 |     // Значение по умолчанию для следующих parallel-областей.
29 |     omp_set_num_threads(2);
30 |
31 |     std::printf("\n2) Region with num_threads(3)\n");
32 | #pragma omp parallel num_threads(3)
33 |     {
34 |         std::printf("Region 1, thread %d of %d\n",
35 |                     omp_get_thread_num(), omp_get_num_threads());
36 |     }
37 |
38 |     std::printf("\n3) Region using omp_set_num_threads(2)\n");
39 | #pragma omp parallel
40 |     {
41 |         std::printf("Region 2, thread %d of %d\n",
42 |                     omp_get_thread_num(), omp_get_num_threads());
```

```
43 |     }
44 |
45 |     return 0;
46 | }
```

## Построчный разбор

Пустые строки и парные фигурные скобки объединены с конструкцией, к которой они относятся; все содержательные строки разобраны ниже.

| Строки | Что происходит и почему |
|---|---|
| 1 | Подключает std::printf для компактного форматированного вывода. |
| 2 | Подключает объявления функций OpenMP. Без omp.h компилятор не знает сигнатуры omp_get_thread_num и других функций. |
| 4 | Точка входа программы. |
| 5–10 | Условная компиляция. При включенном OpenMP печатается значение _OPENMP; иначе выводится ошибка и программа завершается с кодом 1. |
| 12–13 | Отключается динамическое уменьшение команды потоков, чтобы результаты num_threads были предсказуемее. |
| 15 | Заголовок первого эксперимента. |
| 16–17 | Создается parallel-область. Один и тот же блок выполняется всеми потоками команды. |
| 18 | Каждый поток получает собственный идентификатор от 0 до количества потоков минус 1. |
| 19 | Каждый поток печатает свой идентификатор. Порядок строк не определен планировщиком. |
| 21–25 | master оставляет блок только потоку 0; он узнает размер текущей команды через omp_get_num_threads. У master нет неявного барьера. |
| 26 | В конце parallel есть неявный барьер: главный поток продолжит последовательный код после завершения всей команды. |
| 28–29 | Устанавливается размер команды по умолчанию для будущих областей без num_threads. |
| 31–36 | num_threads(3) переопределяет значение 2 только для этой области. Каждый из трех потоков печатает id и фактический размер команды. |
| 38–43 | Во второй области нет num_threads, поэтому среда использует значение 2 из omp_set_num_threads. |
| 45–46 | Код 0 означает успешное завершение. |

## Компиляция и запуск

Linux с GCC:

```
g++ -std=c++17 -O2 -Wall -Wextra -fopenmp lab1_openmp_intro.cpp -o lab1
./lab1
```

macOS с Apple Clang и установленной через Homebrew библиотекой libomp:

```
brew install libomp
clang++ -std=c++17 -O2 -Wall -Wextra -Xpreprocessor -fopenmp \
  -I$(brew --prefix libomp)/include lab1_openmp_intro.cpp \
  -L$(brew --prefix libomp)/lib -lomp -o lab1
./lab1
```

## Что должно получиться

В первой области число строк Thread id равно фактическому числу потоков. Во второй области будет три строки Region 1, в третьей — две строки Region 2. Порядок идентификаторов может меняться от запуска к запуску; это нормально.

## Вопросы для защиты

- Почему без -fopenmp директивы могут быть проигнорированы? — Это pragma, а обработка OpenMP включается отдельным ключом компилятора.
- Чем master отличается от single? — master выполняет только поток 0 и не ставит барьер; single выбирает любой один поток и обычно имеет барьер.
- Что сильнее: omp_set_num_threads(2) или num_threads(3)? — Для конкретной области сильнее num_threads(3).

# Лабораторная работа 2 Суммирование элементов массива

**Цель.** Показать гонку данных при общей переменной sum и исправить ее тремя способами: reduction, atomic и critical.

**Результат.** Массив из 1 000 000 единиц имеет ожидаемую сумму 1 000 000. Ошибочная версия запускается пять раз; три корректные версии всегда возвращают ожидаемое значение.

Файл решения: lab2_array_sum.cpp

## Почему неправильная сумма меняется

Операция sum += a[i] не является одной неделимой операцией. Она состоит из чтения sum, вычисления нового значения и записи. Два потока могут прочитать одинаковое старое значение, увеличить его независимо и записать два результата, один из которых затрет другой. Это гонка данных и, с точки зрения модели памяти C++, неопределенное поведение. Иногда ответ случайно может оказаться правильным — это не делает код корректным.

## Сравнение трех исправлений

| Способ | Механизм и выбор |
|---|---|
| reduction(+:sum) | Лучший вариант для суммы. Каждый поток накапливает локальную копию, а OpenMP объединяет копии в конце. |
| atomic | Подходит для одной операции sum += local_sum. В решении атомарное обновление выполняется один раз на поток, а не на каждый элемент. |
| critical | Может защищать целый блок, но остальные потоки ждут. Для одного сложения обычно тяжелее atomic и reduction. |

## Полный код с номерами строк

```
 1 | #include <cstddef>
 2 | #include <iostream>
 3 | #include <vector>
 4 | #include <omp.h>
 5 |
 6 | int main() {
 7 |     const std::size_t n = 1'000'000;
 8 |     const std::vector<int> a(n, 1);
 9 |     const long long expected = static_cast<long long>(n);
10 |
11 |     std::cout << "Array size: " << n << "\n";
12 |     std::cout << "Expected sum: " << expected << "\n\n";
13 |
14 |     // Учебно ошибочный вариант: несколько потоков одновременно меняют sum.
15 |     for (int attempt = 1; attempt <= 5; ++attempt) {
16 |         long long sum = 0;
17 | #pragma omp parallel for shared(sum)
18 |         for (std::size_t i = 0; i < n; ++i) {
19 |             sum += a[i];
20 |         }
21 |         std::cout << "Race attempt " << attempt << ": " << sum << "\n";
22 |     }
23 |
24 |     long long sum = 0;
25 | #pragma omp parallel for reduction(+:sum)
26 |     for (std::size_t i = 0; i < n; ++i) {
27 |         sum += a[i];
28 |     }
29 |     std::cout << "\nReduction sum: " << sum << "\n";
30 |
31 |     sum = 0;
32 | #pragma omp parallel
33 |     {
34 |         long long local_sum = 0;
35 | #pragma omp for
```

```
36 |         for (std::size_t i = 0; i < n; ++i) {
37 |             local_sum += a[i];
38 |         }
39 | #pragma omp atomic
40 |         sum += local_sum;
41 |     }
42 |     std::cout << "Atomic sum: " << sum << "\n";
43 |
44 |     sum = 0;
45 | #pragma omp parallel
46 |     {
47 |         long long local_sum = 0;
48 | #pragma omp for
49 |         for (std::size_t i = 0; i < n; ++i) {
50 |             local_sum += a[i];
51 |         }
52 | #pragma omp critical(sum_update)
53 |         {
54 |             sum += local_sum;
55 |         }
56 |     }
57 |     std::cout << "Critical sum: " << sum << "\n";
58 |
59 |     return sum == expected ? 0 : 1;
60 | }
```

## Построчный разбор

Пустые строки и парные фигурные скобки объединены с конструкцией, к которой они относятся; все содержательные строки разобраны ниже.

| Строки | Что происходит и почему |
|---|---|
| 1–4 | Подключаются тип размеров, потоки ввода-вывода, vector и функции OpenMP. |
| 6 | Начало программы. |
| 7 | Размер массива. Он достаточно велик, чтобы гонка обычно была заметна. |
| 8 | Конструктор vector создает n элементов и заполняет каждый единицей. |
| 9 | Эталонная сумма равна числу элементов. long long выбран с запасом от переполнения. |
| 11–12 | Печатаются параметры проверки. |
| 14–16 | Начинаются пять повторов намеренно ошибочного опыта; sum каждый раз обнуляется. |
| 17 | parallel for одновременно создает потоки и делит итерации. shared(sum) явно отмечает одну общую переменную. |
| 18–20 | Несколько потоков выполняют sum += a[i] без синхронизации. Потерянные обновления дают неправильный результат. |
| 21–22 | Печать каждого опыта позволяет увидеть неповторяемость гонки. |
| 24 | Новая общая сумма для корректного метода. |
| 25 | reduction(+:sum) создает личный sum=0 для каждого потока и складывает личные суммы при завершении. |
| 26–29 | Итерации распределены между потоками, а результат после барьера уже безопасно объединен и печатается. |
| 31–34 | Перед новой parallel-областью sum обнуляется. local_sum объявлена внутри, поэтому каждый поток имеет отдельную копию. |

| Строки | Что происходит и почему |
|---|---|
| 35–38 | omp for делит цикл; каждый поток накапливает свою сумму без общей записи на каждой итерации. |
| 39–40 | atomic безопасно прибавляет одну local_sum к общей sum. Атомарных операций столько, сколько потоков. |
| 41–42 | После завершения команды итог печатается в последовательной области. |
| 44–51 | Для critical повторяется схема локального накопления. |
| 52–55 | Именованная критическая секция sum_update по одному пропускает потоки к общей sum. |
| 56–57 | После неявного барьера печатается корректный итог. |
| 59–60 | Программа возвращает 0 только если последняя контрольная сумма совпала с ожидаемой. |

## Компиляция и запуск

Linux с GCC:

```
g++ -std=c++17 -O2 -Wall -Wextra -fopenmp lab2_array_sum.cpp -o lab2
./lab2
```

macOS с Apple Clang и установленной через Homebrew библиотекой libomp:

```
brew install libomp
clang++ -std=c++17 -O2 -Wall -Wextra -Xpreprocessor -fopenmp \
  -I$(brew --prefix libomp)/include lab2_array_sum.cpp \
  -L$(brew --prefix libomp)/lib -lomp -o lab2
./lab2
```

## Что записать в отчет

- Пять значений Race attempt и объяснение lost update: чтение–изменение–запись пересекаются между потоками.
- Три одинаковых корректных результата Reduction sum, Atomic sum и Critical sum.
- Вывод: для простой суммы reduction яснее и обычно эффективнее; atomic и critical демонстрируют механизмы синхронизации.

# Лабораторная работа 3 Поиск минимального и максимального значения

**Цель.** Найти минимум и максимум так, чтобы каждый поток работал со своими локальными значениями, и измерить время для 1–10 потоков.

**Результат.** Корректность проверяется на 10 числах с ожидаемыми min = −11 и max = 42.
Производительность измеряется на 20 000 000 элементов.

Файл решения: lab3_min_max.cpp

## Почему reduction удовлетворяет условию

В reduction(min:minimum) reduction(max:maximum) среда OpenMP создает для каждого потока отдельные копии minimum и maximum. Внутри цикла поток изменяет только их. После цикла OpenMP объединяет локальные значения в глобальный минимум и максимум. Поэтому одновременной ручной записи разных потоков в общие minimum и maximum нет.

## Алгоритм

1. Инициализировать минимум самым большим int, максимум — самым маленьким int.
2. Запомнить start = omp_get_wtime().
3. Создать requested_threads потоков с двумя reduction.
4. Статически распределить индексы и обновлять локальные min/max.
5. После объединения локальных результатов получить finish и вернуть значения, время и фактическое число потоков.
6. На большом векторе выполнить по три измерения для каждого числа потоков от 1 до 10 и сохранить лучшее время.

## Полный код с номерами строк

```
 1 | #include <algorithm>
 2 | #include <cstddef>
 3 | #include <iomanip>
 4 | #include <iostream>
 5 | #include <limits>
 6 | #include <utility>
 7 | #include <vector>
 8 | #include <omp.h>
 9 |
10 | struct MinMaxResult {
11 |     int minimum;
12 |     int maximum;
13 |     int actual_threads;
14 |     double seconds;
15 | };
16 |
17 | MinMaxResult find_min_max(const std::vector<int>& values,
18 |                           int requested_threads) {
19 |     int minimum = std::numeric_limits<int>::max();
20 |     int maximum = std::numeric_limits<int>::lowest();
21 |     int actual_threads = 1;
22 |     const long long n = static_cast<long long>(values.size());
23 |
24 |     const double start = omp_get_wtime();
25 | #pragma omp parallel num_threads(requested_threads) \
26 |     reduction(min:minimum) reduction(max:maximum)
27 |     {
28 | #pragma omp master
29 |         actual_threads = omp_get_num_threads();
30 |
31 | #pragma omp for schedule(static)
32 |         for (long long i = 0; i < n; ++i) {
33 |             const int value = values[static_cast<std::size_t>(i)];
34 |             minimum = std::min(minimum, value);
35 |             maximum = std::max(maximum, value);
```

```
36 |         }
37 |     }
38 |     const double finish = omp_get_wtime();
39 |
40 |     return {minimum, maximum, actual_threads, finish - start};
41 | }
42 |
43 | int main() {
44 |     omp_set_dynamic(0);
45 |
46 |     const std::vector<int> check_values = {8, -4, 15, 0, 3,
47 |                                             42, -11, 7, 9, 1};
48 |     const auto [sequential_min, sequential_max] =
49 |         std::minmax_element(check_values.begin(), check_values.end());
50 |     const MinMaxResult check = find_min_max(check_values, 4);
51 |
52 |     std::cout << "Check on 10 elements\n";
53 |     std::cout << "Expected min/max: " << *sequential_min << " / "
54 |               << *sequential_max << "\n";
55 |     std::cout << "OpenMP min/max:   " << check.minimum << " / "
56 |               << check.maximum << "\n\n";
57 |
58 |     const std::size_t n = 20'000'000;
59 |     std::vector<int> values(n);
60 |     for (std::size_t i = 0; i < n; ++i) {
61 |         const long long generated =
62 |             (static_cast<long long>(i) * 48'271LL) % 2'000'001LL;
63 |         values[i] = static_cast<int>(generated - 1'000'000LL);
64 |     }
65 |
66 |     std::cout << "Benchmark vector size: " << n << "\n";
67 |     std::cout << std::fixed << std::setprecision(6);
68 |     std::cout << "threads\ttime, s\tspeedup\tmin\tmax\n";
69 |
70 |     double one_thread_time = 0.0;
71 |     for (int threads = 1; threads <= 10; ++threads) {
72 |         MinMaxResult best{};
73 |         best.seconds = std::numeric_limits<double>::max();
74 |
75 |         for (int repeat = 0; repeat < 3; ++repeat) {
76 |             const MinMaxResult current = find_min_max(values, threads);
77 |             if (current.seconds < best.seconds) {
78 |                 best = current;
79 |             }
80 |         }
81 |
82 |         if (threads == 1) {
83 |             one_thread_time = best.seconds;
84 |         }
85 |         const double speedup = one_thread_time / best.seconds;
86 |         std::cout << best.actual_threads << '\t' << best.seconds << '\t'
87 |                   << speedup << '\t' << best.minimum << '\t'
88 |                   << best.maximum << '\n';
89 |     }
90 |
91 |     return (check.minimum == *sequential_min &&
92 |             check.maximum == *sequential_max) ? 0 : 1;
93 | }
```

## Построчный разбор

Пустые строки и парные фигурные скобки объединены с конструкцией, к которой они относятся; все содержательные строки разобраны ниже.

| Строки | Что происходит и почему |
|---|---|
| 1–8 | Подключаются алгоритмы min/max, типы, форматирование, ввод-вывод, пределы типов, structured binding, vector и OpenMP. |
| 10–15 | Структура упаковывает минимум, максимум, фактическое число потоков и время одного запуска. |
| 17–18 | Функция получает вектор по константной ссылке без копирования и требуемое число потоков. |
| 19–22 | Начальные значения гарантируют, что первый элемент заменит и minimum, и maximum. Размер приводится к long long для канонического OpenMP-цикла. |

| Строки | Что происходит и почему |
|---|---|
| 24 | Начало измеряемого участка перед созданием команды потоков. |
| 25–27 | parallel создает команду, num_threads задает размер, две reduction создают локальные экстремумы каждого потока. |
| 28–29 | Только master записывает фактический размер команды. Это важно, если среда не выполнила запрос буквально. |
| 31 | schedule(static) заранее делит равные по стоимости итерации между потоками. |
| 32–36 | Поток читает свой элемент и сравнивает его только со своими reduction-копиями minimum и maximum. |
| 37–40 | После выхода reduction объединены; замер завершается и функция возвращает результат. |
| 43–44 | В main отключается динамическое уменьшение числа потоков. |
| 46–50 | Малый тест содержит 10 чисел. std::minmax_element создает независимый последовательный эталон, затем вызывается OpenMP-версия. |
| 52–56 | Печатаются эталон и параллельный ответ; должны получиться −11 и 42. |
| 58–64 | Создается 20-миллионный детерминированный вектор в диапазоне примерно от −1 000 000 до 1 000 000. |
| 66–68 | Печатается заголовок таблицы измерений с шестью знаками после запятой. |
| 70–73 | one_thread_time станет базой ускорения; best.seconds начинается с максимально возможного double. |
| 75–80 | Для каждого числа потоков выполняются три повтора и выбирается минимальное время, чтобы уменьшить влияние фоновой нагрузки. |
| 82–88 | Время одного потока сохраняется как $T_1$; затем вычисляется $T_1/T_p$ и печатаются фактические потоки, время, ускорение, min и max. |
| 91–93 | Код возврата подтверждает совпадение OpenMP-результата с эталоном на 10 элементах. |

## Компиляция и запуск

Linux с GCC:

```
g++ -std=c++17 -O2 -Wall -Wextra -fopenmp lab3_min_max.cpp -o lab3
./lab3
```

macOS с Apple Clang и установленной через Homebrew библиотекой libomp:

```
brew install libomp
clang++ -std=c++17 -O2 -Wall -Wextra -Xpreprocessor -fopenmp \
  -I$(brew --prefix libomp)/include lab3_min_max.cpp \
  -L$(brew --prefix libomp)/lib -lomp -o lab3
./lab3
```

## Как анализировать таблицу

Минимум и максимум должны быть одинаковы во всех строках. Время обычно уменьшается сначала заметно, затем медленнее; после числа физических ядер улучшение может исчезнуть. Если время растет, накладные расходы или пропускная способность памяти уже сильнее пользы дополнительных потоков.

# Лабораторная работа 4 Скалярное произведение двух векторов

**Цель.** Вычислить сумму произведений соответствующих элементов двух векторов и исследовать время для 1–10 потоков.

**Результат.** На 10 элементах ожидаемый результат равен 220. Для измерений используются два вектора по 10 000 000 элементов.

Файл решения: lab4_dot_product.cpp

## Математическая идея

Скалярное произведение равно $a_0b_0 + a_1b_1 + \ldots + a_{n-1}b_{n-1}$. Каждая итерация независимо вычисляет одно произведение, а общий результат является суммой, поэтому цикл естественно распараллеливается с reduction(+:result).

Для double порядок сложения влияет на последние биты результата: параллельная сумма группирует слагаемые иначе, чем последовательная. Небольшая разница допустима, поэтому проверка использует абсолютную погрешность, а не точное равенство.

## Полный код с номерами строк

```
 1 | #include <cmath>
 2 | #include <cstddef>
 3 | #include <iomanip>
 4 | #include <iostream>
 5 | #include <limits>
 6 | #include <vector>
 7 | #include <omp.h>
 8 |
 9 | struct DotResult {
10 |     double value;
11 |     int actual_threads;
12 |     double seconds;
13 | };
14 |
15 | DotResult dot_product(const std::vector<double>& a,
16 |                       const std::vector<double>& b,
17 |                       int requested_threads) {
18 |     double result = 0.0;
19 |     int actual_threads = 1;
20 |     const long long n = static_cast<long long>(a.size());
21 |
22 |     const double start = omp_get_wtime();
23 | #pragma omp parallel num_threads(requested_threads) reduction(+:result)
24 |     {
25 | #pragma omp master
26 |         actual_threads = omp_get_num_threads();
27 |
28 | #pragma omp for schedule(static)
29 |         for (long long i = 0; i < n; ++i) {
30 |             const std::size_t index = static_cast<std::size_t>(i);
31 |             result += a[index] * b[index];
32 |         }
33 |     }
34 |     const double finish = omp_get_wtime();
35 |
36 |     return {result, actual_threads, finish - start};
37 | }
38 |
39 | int main() {
40 |     omp_set_dynamic(0);
41 |
42 |     const std::vector<double> check_a = {1, 2, 3, 4, 5,
43 |                                          6, 7, 8, 9, 10};
44 |     const std::vector<double> check_b = {10, 9, 8, 7, 6,
45 |                                          5, 4, 3, 2, 1};
46 |     double expected = 0.0;
47 |     for (std::size_t i = 0; i < check_a.size(); ++i) {
48 |         expected += check_a[i] * check_b[i];
49 |     }
50 |     const DotResult check = dot_product(check_a, check_b, 4);
51 |
52 |     std::cout << "Check on 10 elements\n";
53 |     std::cout << "Expected dot product: " << expected << "\n";
```

```
54 |     std::cout << "OpenMP dot product:   " << check.value << "\n\n";
55 |
56 |     const std::size_t n = 10'000'000;
57 |     std::vector<double> a(n);
58 |     std::vector<double> b(n);
59 |     for (std::size_t i = 0; i < n; ++i) {
60 |         a[i] = 1.0 + static_cast<double>(i % 100) / 100.0;
61 |         b[i] = 2.0 - static_cast<double>(i % 80) / 100.0;
62 |     }
63 |
64 |     std::cout << "Benchmark vector size: " << n << "\n";
65 |     std::cout << std::fixed << std::setprecision(6);
66 |     std::cout << "threads\ttime, s\tspeedup\tdot product\n";
67 |
68 |     double one_thread_time = 0.0;
69 |     for (int threads = 1; threads <= 10; ++threads) {
70 |         DotResult best{};
71 |         best.seconds = std::numeric_limits<double>::max();
72 |
73 |         for (int repeat = 0; repeat < 3; ++repeat) {
74 |             const DotResult current = dot_product(a, b, threads);
75 |             if (current.seconds < best.seconds) {
76 |                 best = current;
77 |             }
78 |         }
79 |
80 |         if (threads == 1) {
81 |             one_thread_time = best.seconds;
82 |         }
83 |         const double speedup = one_thread_time / best.seconds;
84 |         std::cout << best.actual_threads << '\t' << best.seconds << '\t'
85 |                   << speedup << '\t' << best.value << '\n';
86 |     }
87 |
88 |     return std::abs(check.value - expected) < 1e-12 ? 0 : 1;
89 | }
```

## Построчный разбор

Пустые строки и парные фигурные скобки объединены с конструкцией, к которой они относятся; все содержательные строки разобраны ниже.

| Строки | Что происходит и почему |
|---|---|
| 1–7 | Подключаются математические функции, размеры, форматирование, потоки, пределы, vector и OpenMP. |
| 9–13 | Структура возвращает скалярное произведение, фактическое число потоков и время. |
| 15–17 | Функция принимает оба вектора по константным ссылкам и число потоков. |
| 18–20 | Сумма начинается с 0; готовятся фактический размер команды и длина цикла. |
| 22 | Начало измерения. |
| 23–24 | Создается команда потоков, а result становится reduction-переменной суммирования. |
| 25–26 | master сохраняет фактический размер команды. |
| 28–32 | for статически делит индексы. Каждый поток прибавляет произведения к своей локальной копии result. |
| 33–36 | OpenMP объединяет локальные суммы, заканчивается замер, возвращается результат. |
| 39–40 | Начало main и отключение динамического изменения команды. |
| 42–49 | Два тестовых вектора из 10 элементов и независимый последовательный расчет эталона 220. |

| Строки | Что происходит и почему |
|---|---|
| 50–54 | Параллельный расчет на четырех потоках и печать обоих значений. |
| 56–62 | Создаются большие векторы. Формулы дают повторяемые вещественные значения без случайного генератора. |
| 64–66 | Заголовок таблицы измерений. |
| 68–78 | Для 1–10 потоков выполняются три повтора и выбирается лучшее время. |
| 80–85 | Фиксируется $T_1$, вычисляется ускорение и печатаются потоки, время, ускорение и значение произведения. |
| 88–89 | Малый тест считается успешным при абсолютной ошибке менее $10^{-12}$. |

## Компиляция и запуск

Linux с GCC:

```
g++ -std=c++17 -O2 -Wall -Wextra -fopenmp lab4_dot_product.cpp -o lab4
./lab4
```

macOS с Apple Clang и установленной через Homebrew библиотекой libomp:

```
brew install libomp
clang++ -std=c++17 -O2 -Wall -Wextra -Xpreprocessor -fopenmp \
  -I$(brew --prefix libomp)/include lab4_dot_product.cpp \
  -L$(brew --prefix libomp)/lib -lomp -o lab4
./lab4
```

## Ожидаемое наблюдение

Скалярное произведение требует чтения двух массивов и одной операции умножения со сложением. На больших массивах ограничением часто становится скорость памяти, поэтому после нескольких потоков ускорение перестает расти линейно.

# Лабораторная работа 5 Вычисление числа π

**Цель.** Приблизить интеграл методом средних прямоугольников, распределить N итераций между потоками и сравнить один поток с максимально доступным числом потоков.

**Результат.** Программа читает N, печатает два приближения π, абсолютные погрешности, времена и ускорение.

Файл решения: lab5_pi.cpp

## От формулы к циклу

$$\pi = \int_0^1 \frac{4}{1+x^2}\,dx$$

$$\pi \approx \frac{1}{N}\sum_{i=0}^{N-1} \frac{4}{1+x_i^2}, x_i = (i+0.5)\frac{1}{N}$$

Интервал [0, 1] делится на N частей шириной step = 1/N. Для части i берется середина x = (i + 0.5)·step и вычисляется высота $4/(1+x^2)$. Сумма высот умножается на ширину step. Итерации независимы, а сумму объединяет reduction.

M_PI формально не обязана присутствовать во всех режимах C++. Поэтому строки 6–8 добавляют переносимый запасной вариант только тогда, когда библиотека <cmath> не определила M_PI.

## Полный код с номерами строк

```
 1 | #include <cmath>
 2 | #include <iomanip>
 3 | #include <iostream>
 4 | #include <omp.h>
 5 |
 6 | #ifndef M_PI
 7 | #define M_PI 3.1415 926 535 897 9 323 846
 8 | #endif
 9 |
10 | struct PiResult {
11 |     double value;
12 |     int actual_threads;
13 |     double seconds;
14 | };
15 |
16 | PiResult calculate_pi(long long intervals, int requested_threads) {
17 |     const double step = 1.0 / static_cast<double>(intervals);
18 |     double sum = 0.0;
19 |     int actual_threads = 1;
20 |
21 |     const double start = omp_get_wtime();
22 | #pragma omp parallel num_threads(requested_threads) reduction(+:sum)
23 |     {
24 | #pragma omp master
25 |         actual_threads = omp_get_num_threads();
26 |
27 | #pragma omp for schedule(static)
28 |         for (long long i = 0; i < intervals; ++i) {
29 |             const double x = (static_cast<double>(i) + 0.5) * step;
30 |             sum += 4.0 / (1.0 + x * x);
31 |         }
32 |     }
33 |     const double finish = omp_get_wtime();
34 |
35 |     return {sum * step, actual_threads, finish - start};
36 | }
37 |
38 | int main() {
39 |     long long n = 0;
40 |     std::cout << "Enter N: ";
41 |     if (!(std::cin >> n) || n <= 0) {
42 |         std::cerr << "N must be a positive integer.\n";
43 |         return 1;
44 |     }
```

```
45 |
46 |     omp_set_dynamic(0);
47 |     const int maximum_threads = omp_get_max_threads();
48 |
49 |     const PiResult one_thread = calculate_pi(n, 1);
50 |     const PiResult many_threads = calculate_pi(n, maximum_threads);
51 |
52 |     std::cout << std::fixed << std::setprecision(15);
53 |     std::cout << "M_PI:              " << M_PI << "\n";
54 |     std::cout << "Pi, 1 thread:     " << one_thread.value << "\n";
55 |     std::cout << "Absolute error:   "
56 |               << std::abs(one_thread.value - M_PI) << "\n";
57 |     std::cout << "Time, 1 thread:   " << one_thread.seconds << " s\n\n";
58 |
59 |     std::cout << "Pi, " << many_threads.actual_threads << " threads: "
60 |               << many_threads.value << "\n";
61 |     std::cout << "Absolute error:   "
62 |               << std::abs(many_threads.value - M_PI) << "\n";
63 |     std::cout << "Time, " << many_threads.actual_threads << " threads: "
64 |               << many_threads.seconds << " s\n";
65 |     std::cout << "Speedup:          "
66 |               << one_thread.seconds / many_threads.seconds << "\n";
67 |
68 |     return 0;
69 | }
```

## Построчный разбор

Пустые строки и парные фигурные скобки объединены с конструкцией, к которой они относятся; все содержательные строки разобраны ниже.

| Строки | Что происходит и почему |
|---|---|
| 1–4 | Подключаются математические функции и M_PI, форматирование, ввод-вывод и OpenMP. |
| 6–8 | Если реализация не определила M_PI, задается эквивалентная константа. Если M_PI уже есть, этот блок ничего не меняет. |
| 10–14 | Структура результата хранит π, фактическое число потоков и время. |
| 16 | Функция получает число интервалов N и запрошенный размер команды. |
| 17 | Ширина каждого прямоугольника равна 1/N; приведение к double предотвращает целочисленное деление. |
| 18–19 | Сумма высот начинается с нуля, фактическое число потоков — с безопасного значения 1. |
| 21 | Начало измерения. |
| 22–23 | Создается команда, а sum включается в reduction по сложению. |
| 24–25 | master запоминает фактический размер команды. |
| 27–31 | for делит N интервалов. x — середина интервала; $4/(1+x^2)$ прибавляется к локальной сумме потока. |
| 32–35 | После объединения reduction измерение заканчивается; сумма высот умножается на step и возвращается как π. |
| 38–44 | main читает N и отвергает ноль, отрицательное значение или нечисловой ввод. |
| 46–47 | Отключается динамическое уменьшение команды; omp_get_max_threads сообщает стандартный максимальный размер команды для текущего запуска. |
| 49–50 | Один и тот же N рассчитывается сначала одним, затем максимально доступным числом потоков. |

| Строки | Что происходит и почему |
|---|---|
| 52–57 | Печатаются M_PI, приближение одного потока, абсолютная ошибка и время. |
| 59–66 | Печатаются многопоточный результат, ошибка, фактическое число потоков, время и ускорение $T_1/T_p$. |
| 68–69 | Успешное завершение программы. |

## Компиляция и запуск

Linux с GCC:

```
g++ -std=c++17 -O2 -Wall -Wextra -fopenmp lab5_pi.cpp -o lab5
./lab5
```

macOS с Apple Clang и установленной через Homebrew библиотекой libomp:

```
brew install libomp
clang++ -std=c++17 -O2 -Wall -Wextra -Xpreprocessor -fopenmp \
  -I$(brew --prefix libomp)/include lab5_pi.cpp \
  -L$(brew --prefix libomp)/lib -lomp -o lab5
./lab5
```

После запуска введите положительное целое N, например 100 000 000. Значение 1 000 000 000 из задания допустимо, но расчет может занять заметное время.

## Как интерпретировать результат

- При увеличении N ошибка метода средних прямоугольников обычно уменьшается примерно как $1/N^2$, пока не начинают доминировать округления double.
- Однопоточное и многопоточное значения могут различаться в последних знаках из-за другого порядка сложения.
- Максимальное число потоков не гарантирует минимальное время: при недостаточно большом N накладные расходы могут перевесить пользу параллелизма.

# Методика измерений и ответы для защиты

## Что обязательно включить в отчет

- Цель работы и краткое описание алгоритма.
- Полный исходный код и команду компиляции с -fopenmp.
- Контрольный результат на малом наборе данных.
- Таблицу: число потоков, время, ускорение; для лабораторной 5 — еще значение π и абсолютную ошибку.
- Краткий вывод о том, до какого числа потоков ускорение росло и почему затем замедлилось.

## Типичные ошибки

| Ошибка | Почему и как исправить |
|---|---|
| Нет -fopenmp | Директивы игнорируются или не связываются функции omp_*. Всегда проверяйте команду сборки и _OPENMP. |
| Общая sum без reduction | Возникает гонка данных. Результат нельзя считать надежным даже если один запуск совпал. |
| sum не обнулена | Следующая parallel-область начинает с результата предыдущей и получает завышенную сумму. |
| Замер включает заполнение | Сравниваются не только параллельные алгоритмы. В решениях данные создаются до omp_get_wtime. |
| Слишком маленький массив | Время создания потоков больше полезной работы, поэтому параллельная версия медленнее. |
| Сравнение double через == | Порядок сложения меняется. Используйте абсолютную или относительную погрешность. |
| Число потоков считают равным запросу | Среда может изменить размер команды. Для отчета печатайте omp_get_num_threads внутри parallel. |

## Краткие ответы

| Вопрос | Ответ |
|---|---|
| Почему parallel сам не делит цикл? | Потому что parallel только создает команду; без omp for каждый поток выполнит весь обычный цикл. |
| Где возникает барьер? | В конце parallel и, если не указан nowait, в конце omp for. У master барьера нет. |
| Почему reduction быстрее critical? | Локальные копии обновляются без блокировки на каждой итерации, а объединение выполняется один раз в конце. |
| Почему 10 потоков не дают ускорение 10? | Есть последовательные части, накладные расходы, конкуренция за память и ограничение физическими ядрами. |
| Что показывает omp_get_max_threads? | Максимальный стандартный размер команды для следующей области в текущих настройках, а не обещание ускорения. |
| Почему schedule(static)? | Все итерации этих задач имеют почти одинаковую стоимость, поэтому заранее равное распределение имеет минимальные накладные расходы. |

## Использованные материалы

Учебное пособие OpenMP: примеры 1a, 2a, 3a, 4a, 5a, 17a, 18a, 26a и 27a; пять файлов заданий OpenMP 1–5. Код решений написан на C++17 с учетом формулировок заданий и современных средств OpenMP.

## Комплект готовых файлов

| Файл | Назначение |
|---|---|
| lab1_openmp_intro.cpp | Проверка OpenMP, идентификаторы потоков, master, omp_set_num_threads и num_threads. |
| lab2_array_sum.cpp | Гонка данных и корректные суммы через reduction, atomic и critical. |
| lab3_min_max.cpp | Минимум и максимум с reduction и измерениями для 1–10 потоков. |
| lab4_dot_product.cpp | Скалярное произведение и измерения для 1–10 потоков. |
| lab5_pi.cpp | Интегральное приближение π, сравнение одного и максимального числа потоков. |
| Руководство DOCX и PDF | Теория, весь код, построчный разбор, команды сборки и материалы для защиты. |

## Порядок подготовки к сдаче

- Скомпилировать каждый файл с поддержкой OpenMP.
- Сохранить реальные результаты запуска на своем компьютере.
- Для лабораторных 3 и 4 перенести таблицы времени 1–10 потоков в отчет.
- Для лабораторной 5 указать N, погрешность, времена и ускорение.
- Сформулировать вывод по фактическим данным, не ожидая идеального линейного ускорения.